CAHIER DES CHARGES

Projet Fil Rouge – Plateforme SkillHub

Bachelor Concepteur Développeur Web Full Stack

Présentation du projet SkillHub

SkillHub est une plateforme web collaborative destinée à mettre en relation des formateurs et des apprenants autour de formations en ligne.

La plateforme permet aux formateurs de créer et gérer des formations structurées en modules, tandis que les apprenants peuvent découvrir des formations, s'y inscrire et suivre les contenus pédagogiques proposés.

L'objectif principal de SkillHub est de faciliter le partage de compétences et l'apprentissage en ligne, tout en offrant une expérience simple et interactive aux utilisateurs.

Dans le cadre de ce projet fil rouge, les étudiants devront concevoir et développer une application web complète, comprenant :

- une interface utilisateur moderne développée avec React.js
- une API REST sécurisée avec authentification JWT
- une base de données relationnelle pour les données principales
- une base MongoDB pour l'historisation et les logs
- des tests unitaires pour garantir la qualité du code

Ce projet permettra aux étudiants de mettre en pratique les compétences essentielles du développement Full Stack, notamment :

- développement front-end
- conception d'API REST
- gestion de bases de données
- authentification et sécurité
- bonnes pratiques de développement et tests

PARTIE I– FRONT-END

1. Objectif du Front-End

Le front-end de la plateforme **SkillHub** devra être développé en **React.js**.

L'application devra permettre aux utilisateurs de :

- découvrir la plateforme
- consulter les formations disponibles
- s'inscrire en tant qu'apprenant ou formateur
- accéder à leur tableau de bord
- suivre une formation

L'interface devra être **responsive**, claire et structurée en **composants React réutilisables**.

Certaines actions pourront être réalisées via des **fenêtres modales (modals)**, notamment :

- inscription
- connexion
- création ou modification de formation
- gestion des modules

Dans le cadre de ce projet pédagogique, **toutes les formations sont gratuites**.

---

2. Pages principales de l'application

L'application devra comporter les pages suivantes :

1. Page d'accueil
2. Page des formations
3. Page détail de formation
4. Tableau de bord apprenant
5. Tableau de bord formateur
6. Page de suivi de formation

---

3. Page d'accueil

Objectif

La page d'accueil est le **point d'entrée principal de la plateforme**.

Elle doit permettre aux utilisateurs de comprendre le concept de SkillHub et de s'inscrire ou se connecter.

Contenu de la page

Barre de navigation

La navbar doit contenir :

- logo ou nom de la plateforme
- lien vers **Formations**
- bouton **Se connecter**
- bouton **S'inscrire**

Si l'utilisateur est connecté :

- affichage du **profil utilisateur**

En cliquant sur le profil, l'utilisateur est redirigé vers son **tableau de bord** selon son rôle.

Présentation de la plateforme

Section présentant :

- le concept de SkillHub
- l'objectif de la plateforme
- les avantages pour les apprenants et formateurs

Inscription avec choix du rôle

Lors de l'inscription, l'utilisateur doit choisir :

- **Apprenant**
- **Formateur**

Ce choix détermine le type de **dashboard accessible après connexion**.

L'inscription peut être réalisée via une **modal**.

Aperçu des formations

La page d'accueil doit afficher **quelques formations mises en avant**.

Chaque formation affiche :

- titre
- niveau
- formateur

Un bouton **Voir toutes les formations** redirige vers la page des formations.

4. Page des formations

Objectif

Permet aux utilisateurs de consulter toutes les formations disponibles.

Système de filtres

La page doit contenir au minimum **trois filtres** :

Recherche

Permet de rechercher une formation par mot-clé.

Filtre par catégorie

Exemples :

- Développement web
- Data
- Design
- Marketing
- DevOps

Filtre par niveau

Niveaux possibles :

- Débutant
- Intermédiaire
- Avancé

Affichage des formations

Les formations doivent être affichées sous forme de **cards**.

Chaque card doit contenir :

- titre
- mini description
- niveau

- nombre d’apprenants qui suivent ou ont suivi la formation
- nombre de vues
- bouton **Voir détail**

---

5. Page détail de formation

Objectif

Afficher les informations complètes d’une formation.

Contenu

La page doit afficher :

- titre de la formation
- description complète
- catégorie
- niveau
- formateur
- nombre d’apprenants
- nombre de vues

Liste des modules

La page doit afficher la liste des modules de la formation.

Exemple :

Module 1 – Introduction
Module 2 – Concepts fondamentaux
Module 3 – Projet pratique

Chaque formation doit contenir **au minimum 3 modules**.

Actions possibles

<u>Utilisateur non connecté</u>

Bouton :

Suivre la formation

Ce bouton redirige vers la page / modal de connextion.

Utilisateur déjà inscrit

Bouton :

Suivre la formation

Ce bouton redirige vers la page de suivi.

---

6. Tableau de bord formateur

Objectif

Permet au formateur de gérer les formations qu'il a créées.

Création de formation

Un bouton en haut du dashboard :

+ **Créer une formation**

ouvre une **modal de création de formation**.

Affichage des formations

Les formations du formateur sont affichées sous forme de **cards**.

Chaque card contient :

- titre
- mini description
- nombre de vues
- nombre d'apprenants

Actions disponibles

Chaque card doit contenir trois boutons :

Voir détail

Permet d’accéder à la page détail de la formation.

Modifier

Permet de modifier les informations de la formation.

La modification peut se faire via une **modal**.

Supprimer

Permet de supprimer la formation.

Une confirmation doit être demandée avant suppression.

---

7. Tableau de bord apprenant

Objectif

Permet à l’apprenant de gérer les formations auxquelles il est inscrit.

Affichage des formations

Les formations suivies sont affichées sous forme de **cards**.

Chaque card contient :

- titre
- mini description
- niveau
- progression de la formation (optionnel)

Actions disponibles

Chaque card doit contenir deux boutons.

Suivre

Permet d’accéder à la page de suivi de formation.

Ne plus suivre

Permet de se désinscrire de la formation.

Une confirmation doit être demandée.

8. Page de suivi de formation

Objectif

Permet à un apprenant inscrit de suivre le contenu pédagogique d'une formation.

Accès à la page

L'utilisateur peut accéder à cette page :

- depuis son **dashboard apprenant**
- depuis la **page détail de formation**

Contenu

La page doit afficher :

- titre de la formation
- description
- liste des modules

Chaque module peut contenir :

- texte
- vidéo
- ressources pédagogiques

Progression

La page doit afficher la **progression de l'apprenant** dans la formation.

---

9. Navigation (React Router)

Routes principales :

```
/                   → Page d'accueil
/formations         → Liste des formations
/formation/:id      → Détail formation
/dashboard/apprenant → Dashboard apprenant
/dashboard/formateur → Dashboard formateur
/apprendre/:id      → Suivi de formation
```

10. Contraintes techniques

Le front-end devra respecter les contraintes suivantes :

- développement en **React.js**
- architecture en **composants**
- utilisation de **React Router**
- gestion des appels API avec **Fetch ou Axios**
- interface **responsive**
- code structuré et lisible

PARTIE II– BACK-END

1. Objectif du Back-End

Le back-end de la plateforme **SkillHub** doit fournir une **API REST** permettant au front-end d'interagir avec les données de l'application.

Le backend devra gérer :

- l'authentification des utilisateurs
- la gestion des rôles (apprenant / formateur)
- la gestion des formations
- la gestion des modules de formation
- l'inscription aux formations
- le suivi des apprenants
- les statistiques (vues, nombre d'apprenants)

2. Technologie

Les étudiants sont libres de choisir le **framework backend** qu'ils souhaitent utiliser pour développer l'API REST.

Exemples possibles :

- Node.js (Express / NestJS)
- Symfony
- Django / Django REST Framework
- Laravel
- Spring Boot
- Flask

Quel que soit le framework choisi, l'API devra respecter les **principes REST**.

---

3. Authentification

L'authentification devra obligatoirement être implémentée avec **JWT (JSON Web Token)**.

Le token JWT devra être utilisé pour sécuriser les routes protégées de l'API.

Fonctionnalités attendues :

- inscription d'un utilisateur
- connexion
- génération d'un token JWT
- accès aux routes protégées avec le token

Exemple d'utilisation :

```
Authorization: Bearer <token>
```

---

4. Base de données

L'application devra utiliser une **base de données relationnelle**.

Exemples possibles :

- PostgreSQL
- MySQL

---

5. Modèle de données principal

Le système devra gérer les entités suivantes.

Utilisateur (User)

Représente un utilisateur de la plateforme.

Champs possibles :

- id
- nom
- email
- mot_de_passe
- rôle (apprenant ou formateur)

- date_creation

Formation (Course)

Représente une formation créée par un formateur.

Champs possibles :

- id
- titre
- description
- catégorie
- niveau
- formateur_id
- nombre_de_vues
- date_creation

Module

Une formation est composée de plusieurs modules.

Champs possibles :

- id
- titre
- contenu
- formation_id
- ordre

Chaque formation doit contenir **au minimum 3 modules**.

Inscription (Enrollment)

Représente l'inscription d'un apprenant à une formation.

Champs possibles :

- id
- utilisateur_id
- formation_id
- progression
- date_inscription

---

6. Endpoints principaux de l'API

Authentification

POST

`/api/register`

Créer un compte utilisateur.

POST

`/api/login`

Connexion d'un utilisateur et génération d'un **token JWT**.

GET

`/api/profile`

Récupérer les informations du profil connecté.

Route protégée par JWT.

Formations

GET

`/api/formations`

Récupérer la liste des formations.

Support des filtres :

- recherche
- catégorie
- niveau

GET

`/api/formations/{id}`

Récupérer les détails d'une formation.

POST

`/api/formations`

Créer une formation (formateur uniquement).

PUT

`/api/formations/{id}`

Modifier une formation.

DELETE

`/api/formations/{id}`

Supprimer une formation.

Modules

GET

`/api/formations/{id}/modules`

Récupérer les modules d’une formation.

POST

`/api/formations/{id}/modules`

Ajouter un module.

PUT

`/api/modules/{id}`

Modifier un module.

DELETE

`/api/modules/{id}`

Supprimer un module.

Inscription à une formation

POST

`/api/formations/{id}/inscription`

Permet à un apprenant de s'inscrire à une formation.

DELETE

`/api/formations/{id}/inscription`

Permet à un apprenant de ne plus suivre une formation.

GET

`/api/apprenant/formations`

Récupérer les formations suivies par l'apprenant.

---

7. Gestion des vues

Lorsque la page détail d'une formation est consultée, le backend devra :

- incrémenter automatiquement le **nombre de vues** de la formation.

---

8. Gestion des rôles

Le backend devra vérifier les permissions selon le rôle de l'utilisateur.

Formateur

Peut :

- créer une formation
- modifier une formation
- supprimer une formation

- gérer les modules

Apprenant

Peut :

- s'inscrire à une formation
- suivre une formation
- se désinscrire d'une formation

---

9. Format des réponses API

Toutes les réponses de l'API devront être au format **JSON**.

Exemple :

```
{
  "id": 1,
  "titre": "Introduction à React",
  "niveau": "Débutant",
  "vues": 120,
  "apprenants": 35
}
```

---

10. Documentation de l'API

Les étudiants devront fournir une **documentation de leur API**.

Exemples possibles :

- Swagger / OpenAPI
- Postman Collection
- documentation écrite

---

11. Tests unitaires

Les étudiants devront implémenter des **tests unitaires** afin de vérifier le bon fonctionnement des principales fonctionnalités de l'API.

Les tests doivent permettre de valider :

- les endpoints de l'API
- la logique métier
- les opérations sur la base de données

- la gestion des erreurs

Outils possibles

Les étudiants sont libres de choisir l'outil de test en fonction du framework utilisé.

Exemples :

Node.js

- Jest
- Mocha
- Supertest

Symfony / Laravel / PHP

- PHPUnit

Django / Python

- Pytest
- Django Test Framework

Endpoints à tester (minimum)

Les tests devront couvrir au minimum :

Authentification

- connexion d'un utilisateur

Gestion des formations

- création d'une formation

Sécurité

Les tests doivent vérifier :

- qu'un utilisateur **non authentifié** ne peut pas accéder aux routes protégées
- qu'un **apprenant ne peut pas créer une formation**
- qu'un **formateur ne peut modifier que ses propres formations**

Résultat attendu

Les tests doivent pouvoir être exécutés via une commande simple.

Exemples :

```
npm test
```

ou

```
php bin/phpunit
```

ou

```
pytest
```

Les étudiants devront fournir :

- les fichiers de tests
- les instructions pour exécuter les tests

---

12. Historisation et logs avec MongoDB

En complément de la base de données relationnelle utilisée pour les données principales de l'application, le système devra utiliser **MongoDB** afin d'enregistrer certaines actions importantes effectuées sur la plateforme.

MongoDB sera utilisé pour **l'historisation et la traçabilité des actions** des utilisateurs.

Objectif

L'objectif de ce mécanisme est de :

- garder une trace des actions importantes dans la plateforme
- pouvoir analyser l'activité des utilisateurs
- conserver l'historique des modifications des formations

Événements à enregistrer

Les événements suivants devront être enregistrés dans MongoDB.

Consultation d'une formation

Lorsqu'un utilisateur consulte la page **détail d'une formation**, un événement devra être enregistré.

Inscription à une formation

Lorsqu'un apprenant s'inscrit à une formation, un événement devra être enregistré.

Création d'une formation

Lorsqu'un formateur crée une nouvelle formation, cette action peut être enregistrée dans MongoDB.

Modification d'une formation

Chaque modification d'une formation devra être enregistrée dans MongoDB.

Pour cette action, il est **obligatoire de stocker :**

- les valeurs **avant modification**
- les valeurs **après modification**

Cela permet de conserver un historique des changements.

Suppression d'une formation (optionnel)

Si une formation est supprimée, cet événement peut être enregistré dans MongoDB.

Ajout ou modification de modules (optionnel)

Lorsqu'un module est :

- ajouté
- modifié

cet événement peut également être enregistré dans MongoDB.

Exemple de document MongoDB

Exemple pour une **inscription à une formation** :

```
{
  "event": "course_enrollment",
  "user_id": 12,
  "course_id": 5,
  "timestamp": "2026-03-04T10:30:00"
}
```

Exemple pour une **modification de formation** :

```
{
  "event": "course_update",
  "course_id": 5,
  "updated_by": 3,
  "old_values": {
```

```
    "title": "Introduction React",
    "level": "Débutant"
  },
  "new_values": {
    "title": "React pour débutants",
    "level": "Intermédiaire"
  },
  "timestamp": "2026-03-04T11:00:00"
}
```

Collection MongoDB recommandée

Les événements pourront être stockés dans une collection :

```
activity_logs
```

Chaque document représente un **événement enregistré par l'application**.

PARTIE III - RENDU ATTENDU DES ÉTUDIANTS

Dans le cadre du projet **SkillHub**, chaque groupe d'étudiants devra produire et livrer plusieurs éléments permettant d'évaluer la qualité de leur travail.

Le projet devra être réalisé **en groupe** et chaque étudiant devra contribuer activement au développement de l'application.

---

1. Code source du projet

Les étudiants devront fournir l'intégralité du **code source de l'application**.

Le projet devra inclure :

- le front-end développé en React.js
- le back-end développé sous forme d'API REST
- les tests unitaires
- la configuration de la base de données
- l'intégration avec MongoDB pour l'historisation

Le code devra être structuré et respecter les **bonnes pratiques de développement**.

---

2. Dépôt Git

Le projet devra être hébergé dans un **dépôt Git** (GitHub, GitLab ou équivalent).

Le dépôt devra contenir :

- l'historique des commits
- les branches utilisées dans le projet
- l'ensemble du code source

Les commits devront être **clairs et réguliers**.

---

3. Documentation technique

Les étudiants devront fournir une **documentation technique** expliquant :

- l'architecture de l'application
- les technologies utilisées
- la structure du projet
- le fonctionnement de l'API

La documentation devra également inclure :

- les instructions d'installation du projet
- les instructions pour lancer l'application

---

4. Documentation de l'API

Les étudiants devront documenter leur **API REST**.

Cette documentation peut être réalisée avec :

- Swagger / OpenAPI
- Postman
- ou un document écrit décrivant les endpoints

La documentation devra expliquer :

- les endpoints disponibles
- les paramètres attendus
- les réponses retournées

---

5. Tests unitaires

Les étudiants devront implémenter des **tests unitaires** pour vérifier le bon fonctionnement de l'API.

Les tests devront couvrir au minimum :

- l'authentification
- la gestion des formations
- l'inscription aux formations
- la sécurité des routes protégées

---

6. Structure du projet

Le projet devra inclure au minimum les éléments suivants :

- application front-end React
- API REST backend
- base de données relationnelle
- base MongoDB pour les logs
- tests unitaires